NATIONAL TEACHERS COLLEGE
629 J. Nepomuceno Street, Quiapo, Manila, Philippines
SCHOOL OF ARTS, SCIENCES, AND TECHNOLOGY
Bachelor of Science in Information Technology
Academic Term 2025-2026

Week 12: Midterm Examination (Milestone 2)

Employee Management System

Presented by:
Rivera, Bhea R.
3.1 BSIT

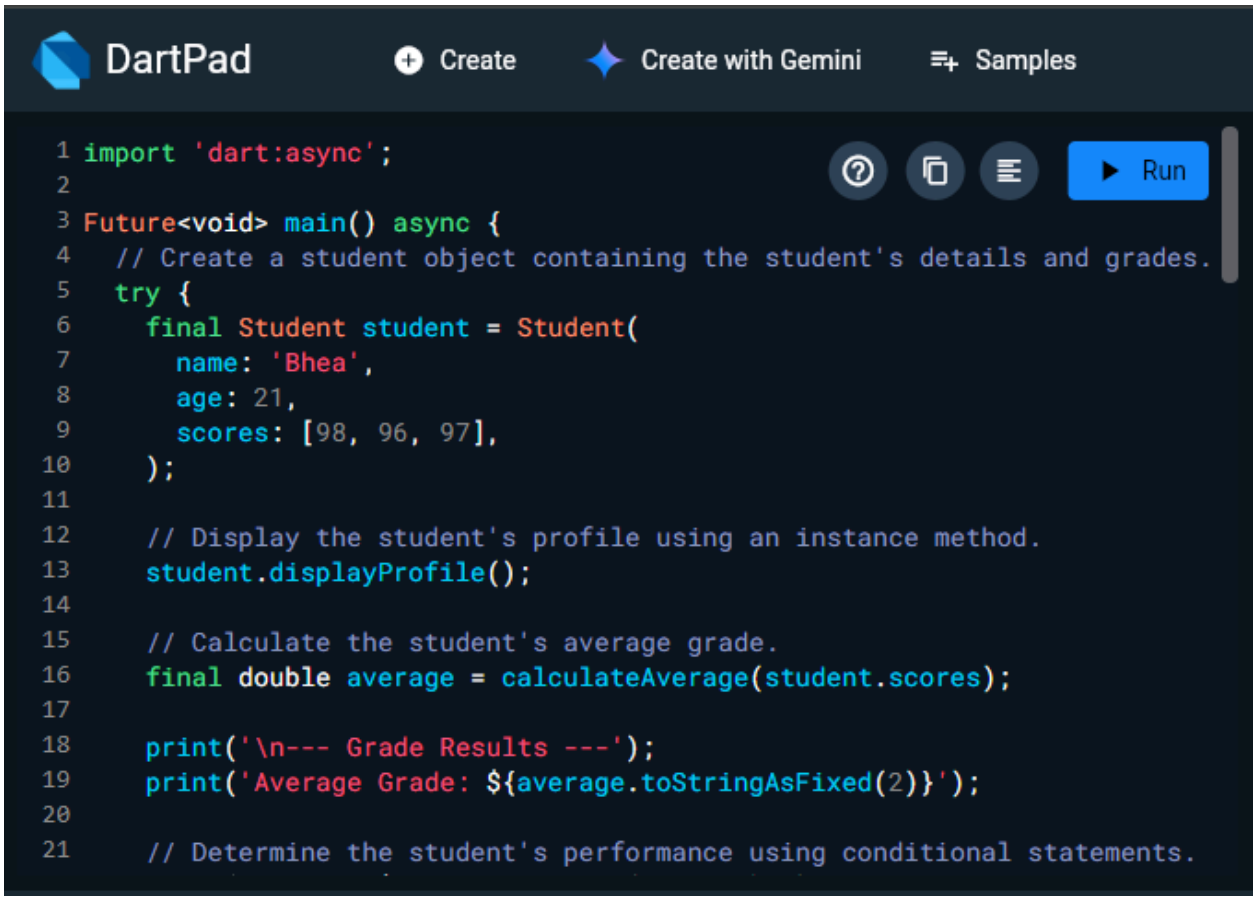
Instructor:
Mr. Edward James V. Grageda

Mobile Development w/Lab

October 27, 2026

NATIONAL TEACHERS COLLEGE
629 J. Nepomuceno Street, Quiapo, Manila, Philippines
SCHOOL OF ARTS, SCIENCES, AND TECHNOLOGY
Bachelor of Science in Information Technology
Academic Term 2025-2026

Code:



NATIONAL TEACHERS COLLEGE
629 J. Nepomuceno Street, Quiapo, Manila, Philippines
SCHOOL OF ARTS, SCIENCES, AND TECHNOLOGY
Bachelor of Science in Information Technology
Academic Term 2025-2026

```
    print('Status: ${getGradeStatus(average)}');

    // Display each grade using a loop.
    print('\n--- Individual Grades ---');
    for (int i = 0; i < student.scores.length; i++) {
      print('Subject ${i + 1}: ${student.scores[i]}');
    }

    // Wait for the asynchronous function to return a message.
    final String message = await getStudyMessage(student.name);
    print('\n$message');
  } catch (error) {
    // Display an error message if something goes wrong.
    print('An error occurred: $error');
  }
}
```

```
// This class stores the student's personal information and grades.
class Student {
  final String name;
  final int age;
  final List<int> scores;

  // Initialize the student and validate the provided information.
  Student({
    required this.name,
    required this.age,
    required this.scores,
  }) {
    if (name.trim().isEmpty) {
      throw ArgumentError('Student name cannot be empty.');
    }

    if (age <= 0) {
      throw ArgumentError('Age must be greater than zero.');
    }

```

NATIONAL TEACHERS COLLEGE
629 J. Nepomuceno Street, Quiapo, Manila, Philippines
SCHOOL OF ARTS, SCIENCES, AND TECHNOLOGY
Bachelor of Science in Information Technology
Academic Term 2025-2026

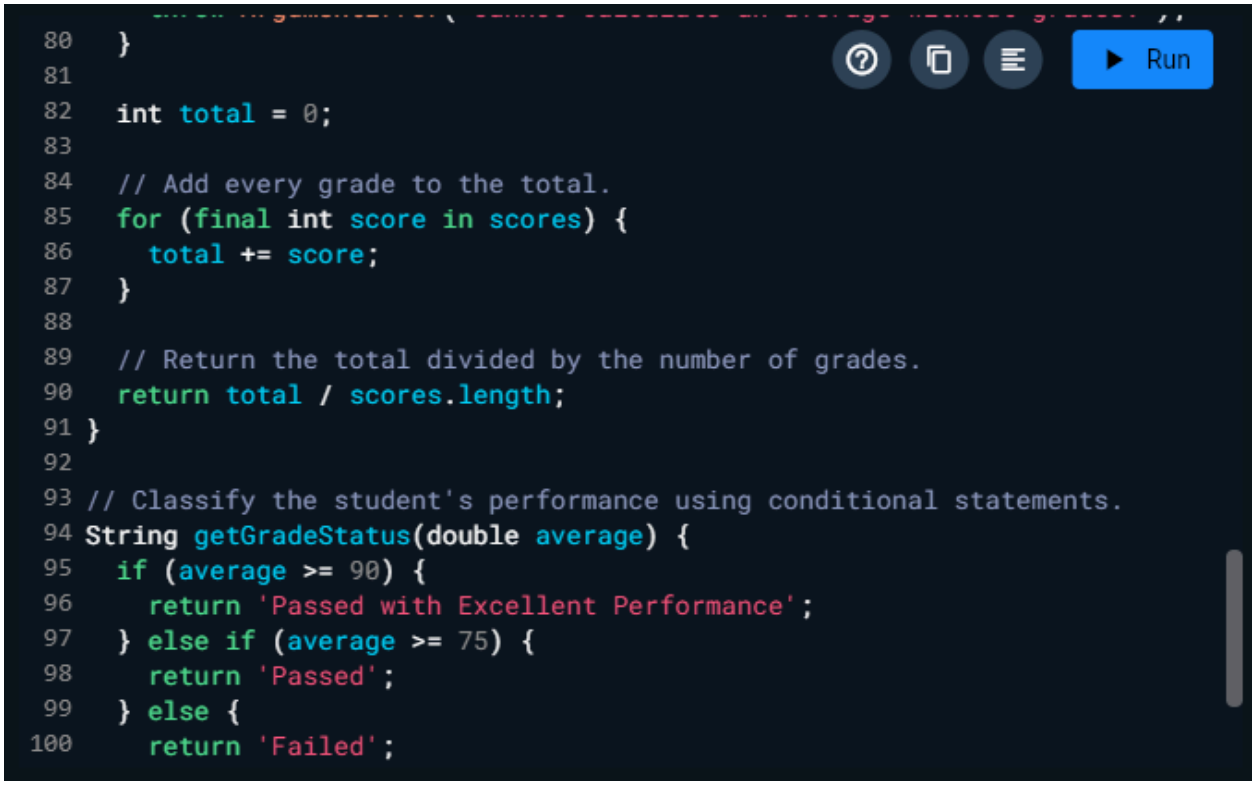
```
    if (scores.isEmpty) {
      throw ArgumentError('At least one grade is required.');
    }

    if (scores.any((score) => score < 0 || score > 100)) {
      throw ArgumentError('Grades must be between 0 and 100.');
    }
  }

  // Display the student's personal information.
  void displayProfile() {
    print('=== Student Profile ===');
    print('Name: $name');
    print('Age: $age');
  }
}

// Calculate and return the average of all grades.
double calculateAverage(List<int> scores) {
  if (scores.isEmpty) {
    throw ArgumentError('Cannot calculate an average without grades.');
```

NATIONAL TEACHERS COLLEGE
629 J. Nepomuceno Street, Quiapo, Manila, Philippines
SCHOOL OF ARTS, SCIENCES, AND TECHNOLOGY
Bachelor of Science in Information Technology
Academic Term 2025-2026




```
  }

  int total = 0;

  // Add every grade to the total.
  for (final int score in scores) {
    total += score;
  }

  // Return the total divided by the number of grades.
  return total / scores.length;
}

// Classify the student's performance using conditional statements.
String getGradeStatus(double average) {
  if (average >= 90) {
    return 'Passed with Excellent Performance';
  } else if (average >= 75) {
    return 'Passed';
  } else {
    return 'Failed';
```



```
  }
}

// Simulate an asynchronous operation that returns a study message.
Future<String> getStudyMessage(String name) async {
  await Future.delayed(const Duration(milliseconds: 300));
  return 'Keep practicing, $name!';
}
```

Output:

```
=== Student Profile ===
Name: Bhea
Age: 21

--- Grade Results ---
Average Grade: 97.00
Status: Passed with Excellent Performance

--- Individual Grades ---
Subject 1: 98
Subject 2: 96
Subject 3: 97

Keep practicing, Bhea!
```

Reflection:

I organized my program using a Student class to keep the student's personal information and grades together. I used a constructor to initialize the student's details and validate the information before processing it. I separated the average calculation and grade classification into functions to make the program easier to understand, test, and maintain. I used conditional statements to determine the student's performance and loops to calculate and display the grades. I included error handling to identify invalid information and prevent incorrect calculations. I also used asynchronous programming with async and await to demonstrate how the program can handle an operation that takes time. Overall, I chose this structure to improve code readability, organization, reliability, and maintainability.